

Data Archivist

Digital Preservation in the Cyber Archives

Summary: Preserve digital knowledge by mastering file operations, managing data streams, and building robust archival systems that protect information.

Version: 3.0

Contents

I Introduction **2**

II AI Instructions **3**

III Common Instructions **5**

III.1 General Rules 5

III.2 Output Specifications 5

IV Exercise 0: Ancient Text Recovery **6**

V Exercise 1: Archive Creation **8**

VI Exercise 2: Stream Management **10**

VII Exercise 3: Vault Security **12**

VIII Turn in and Submission **14**

Chapter I

Introduction

Welcome to the Cyber Archives!

In the year 2087, humanity's greatest treasure is not gold or diamonds: it's **data**. Every piece of digital knowledge, from ancient memes to quantum algorithms, is stored in the vast Cyber Archives. But data is fragile. Without proper preservation, it vanishes into the digital void forever.

That's where you come in. As a Data Archivist, your mission is to master the ancient arts of **file operations** to ensure no knowledge is ever lost.

Your tools are simple but powerful: the ability to **open** data vaults, **read** their contents, **write** new archives, and, most crucially, handle the unexpected without losing precious information.

The Archives are counting on you!

Chapter II

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter III

Common Instructions

III.1 General Rules

- All programs must be written in Python 3.10 or later.
- Your code must comply with the flake8 linter standards.
- All functions and methods must include type hints. Check this using `mypy` .
- Your functions should handle exceptions gracefully to avoid crashes.
- Submit your work to the assigned Git repository.
- Only the content in this repository will be evaluated.



Note for advanced learners: the use of the **with** statement will be introduced in exercise 3. You must not use it before then.



The following standard types and collections are allowed, along with all their associated methods and constructors: str, int, float, list, dict, set, tuple.

III.2 Output Specifications

Each exercise provides terminal examples showing the suggested output format. While the core structure must be maintained, you may customize messages to reflect your understanding of the file operations, as long as the essential information is preserved.

Chapter IV

Exercise 0: Ancient Text Recovery

	Exercise0
ft_ancient_text	
Directory: *ex*0/	
Files to Submit: `ft_ancient_text.py`	
Authorized: `import sys, sys.argv, len(), open(), import typing, typing.IO, io.read(), io.close(), print()`	

Mission Briefing: The Archives have discovered an ancient data fragment in Storage Vault 7. Your first assignment as a Data Archivist is to recover this precious information before it degrades further.

Get the name of a file from the command line, then read the file's contents and display them as the `cat` command would. Add a few headers and footers as shown in the example. You need to handle the various failure cases (nonexistent files, inaccessible files, etc.).



`What is the type of the data returned by open()?`

Example:

```
$> cat ancient_fragment.txt
[FRAGMENT 001] Digital preservation protocols established 2087
[FRAGMENT 002] Knowledge must survive the entropy wars
[FRAGMENT 003] Every byte saved is a victory against oblivion

$> python3 ft_ancient_text.py
Usage: ft_ancient_text.py <file>

$> python3 ft_ancient_text.py foo
=== Cyber Archives Recovery ===
Accessing file 'foo'
Error opening file 'foo': [Errno 2] No such file or directory: 'foo'

$> python3 ft_ancient_text.py /etc/master.passwd
=== Cyber Archives Recovery ===
Accessing file '/etc/master.passwd'
Error opening file '/etc/master.passwd': [Errno 13] Permission denied: '/etc/master.passwd'

$> python3 ft_ancient_text.py ancient_fragment.txt
=== Cyber Archives Recovery ===
Accessing file 'ancient_fragment.txt'
---

[FRAGMENT 001] Digital preservation protocols established 2087
[FRAGMENT 002] Knowledge must survive the entropy wars
[FRAGMENT 003] Every byte saved is a victory against oblivion

---
File 'ancient_fragment.txt' closed.
```

Chapter V

Exercise 1: Archive Creation

	Exercise1
ft_archive_creation	
Directory: *ex*1/	
Files to Submit: `ft_archive_creation.py`	
Authorized: `import sys, sys.argv, len(), open(), import typing, typing.IO, io.read(), io.write(), io.close(), print(), input()`	

Mission Briefing: Excellent work on the data recovery! Your next assignment is to establish a new preservation protocol by creating fresh archive entries.

Use the code created for the previous exercise. At the end of the script, improve the code to:

- Add a special archive character (the `#` character) at the end of each line (let's say, to be 2087-compatible)
- Display the new content
- Ask the user for the name of the file to save to, or leave it empty to avoid saving anything
- Save the new content if a file name is provided and display a new ending message



`Create the file or replace it if it already exists.`

Example:

```
$> python3 ft_archive_creation.py ancient_fragment.txt
=== Cyber Archives Recovery & Preservation ===
Accessing file 'ancient_fragment.txt'
---

[FRAGMENT 001] Digital preservation protocols established 2087
[FRAGMENT 002] Knowledge must survive the entropy wars
[FRAGMENT 003] Every byte saved is a victory against oblivion

---
File 'ancient_fragment.txt' closed.

Transform data:
---

[FRAGMENT 001] Digital preservation protocols established 2087#
[FRAGMENT 002] Knowledge must survive the entropy wars#
[FRAGMENT 003] Every byte saved is a victory against oblivion#

---
Enter new file name (or empty):
Not saving data.
```

```
$> python3 ft_archive_creation.py ancient_fragment.txt
=== Cyber Archives Recovery & Preservation ===
Accessing file 'ancient_fragment.txt'
---

[FRAGMENT 001] Digital preservation protocols established 2087
[FRAGMENT 002] Knowledge must survive the entropy wars
[FRAGMENT 003] Every byte saved is a victory against oblivion

---
File 'ancient_fragment.txt' closed.

Transform data:
---

[FRAGMENT 001] Digital preservation protocols established 2087#
[FRAGMENT 002] Knowledge must survive the entropy wars#
[FRAGMENT 003] Every byte saved is a victory against oblivion#

---
Enter new file name (or empty): new_fragment.txt
Saving data to 'new_fragment.txt'
Data saved in file 'new_fragment.txt'.

$> cat new_fragment.txt
[FRAGMENT 001] Digital preservation protocols established 2087#
[FRAGMENT 002] Knowledge must survive the entropy wars#
[FRAGMENT 003] Every byte saved is a victory against oblivion#
```

Chapter VI

Exercise 2: Stream Management

	Exercise2
ft_stream_management	
Directory: *ex*2/	
Files to Submit: `ft_stream_management.py`	
Authorized: `import sys, sys.argv, sys.stdin, sys.stdout, sys.stderr, len(), open(), import typing, typing.IO, io.read(), io.readline(), io.write(), io.flush(), io.close(), print()`	

Mission Briefing: The Archives operate through three sacred data channels that have been active since the founding of digital civilization. Master these channels that are older than the Internet itself!

Use the code created for the previous exercise. Update it to:

- Print error messages resulting from exceptions to the error output stream instead of to the standard output, with a clear prefix (see example)
- Get user input without using the `input()` built-in function.

Example:

```
$> python3 ft_stream_management.py foo
=== Cyber Archives Recovery & Preservation ===
Accessing file 'foo'
[STDERR] Error opening file 'foo': [Errno 2] No such file or directory: 'foo'
```

```
python3 ft_stream_management.py ancient_fragment.txt
=== Cyber Archives Recovery & Preservation ===
Accessing file 'ancient_fragment.txt'
---

[FRAGMENT 001] Digital preservation protocols established 2087
[FRAGMENT 002] Knowledge must survive the entropy wars
[FRAGMENT 003] Every byte saved is a victory against oblivion

---
File 'ancient_fragment.txt' closed.

Transform data:
---

[FRAGMENT 001] Digital preservation protocols established 2087#
[FRAGMENT 002] Knowledge must survive the entropy wars#
[FRAGMENT 003] Every byte saved is a victory against oblivion#

---
Enter new file name (or empty): /etc/passwd
Saving data to '/etc/passwd'
[STDERR] Error opening file '/etc/passwd': [Errno 13] Permission denied: '/etc/passwd'
Data not saved.
```

Chapter VII

Exercise 3: Vault Security

	Exercise3
ft_vault_security	
Directory: *ex*3/	
Files to Submit: `ft_vault_security.py`	
Authorized: `open(), read(), write(), print()`	

Mission Briefing: The Head Archivist has noticed your potential and is promoting you to Vault Security operations. This is where the real archivists prove themselves.

This exercise requires the use of the `with` statement (context manager) to ensure proper file handling. The `with` statement automatically closes files even if errors occur, preventing resource leaks.

Create a function `secure_archive()` that provides safe access to any file for reading or writing. It returns a tuple `(True|False, str)` that indicates whether the operation succeeded (the boolean) and provides the associated content (either the file's contents or an error message). The function takes the following parameters: a mandatory file name, an optional `int` or `str` (your choice) that indicates the action to perform (read or write), and another optional string that contains the content to write to the file.

During the defense, the structure of the code will be reviewed to match these requirements.

Example:

```
$> python3 ft_vault_security.py
=== Cyber Archives Security ===

Using 'secure_archive' to read from a nonexistent file:
(False, "[Errno 2] No such file or directory: '/not/existing/file'")

Using 'secure_archive' to read from an inaccessible file:
(False, "[Errno 13] Permission denied: '/etc/master.passwd'")

Using 'secure_archive' to read from a regular file:
(True, '[FRAGMENT 001] Digital preservation protocols established 2087\n[FRAGMENT 002] Knowledge must
    survive the entropy wars\n[FRAGMENT 003] Every byte saved is a victory against oblivion\n')

Using 'secure_archive' to write previous content to a new file:
(True, 'Content successfully written to file')
```

Chapter VIII

Turn in and Submission

Turn in your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your files to ensure they are correct.



During evaluation, you may be asked to explain file operations, demonstrate error handling, or show how the **with** statement works. Make sure you understand the concepts behind each exercise.



You need to submit only the files requested by the project specification. Focus on clean, simple code that clearly demonstrates your understanding of file operations.